

Practical - 7

Practical Title: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition: Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

Theory

1. Is the form submitted using POST?

Answer: Yes. The form should use the method="POST" attribute.

When the user clicks the submit button, the form data is sent to the PHP server using the HTTP POST method. In PHP, the submitted values can then be accessed using the \$_POST superglobal.

For example:

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $name = $_POST["name"];  
    $email = $_POST["email"];  
}
```

Using POST is suitable for form submission because the submitted values are sent in the HTTP request body rather than being displayed in the URL

Reference Link:

https://www.w3schools.com/php/php_forms.asp

https://www.w3schools.com/php/php_superglobals_post.asp

2. Are inputs validated and sanitized server side?

Answer: Yes. Server-side validation checks whether the submitted data is correct and follows the required format.

For example:

- Name should not be empty.
- Email should have a valid email format.
- Required fields must be filled.
- Invalid or unwanted characters should be handled safely.

A common PHP approach is:

```
$name = trim($_POST["name"]);  
$email = trim($_POST["email"]);
```

```

if (empty($name)) {
    $error = "Name is required";
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $error = "Invalid email address";
}

```

PHP's `filter_var()` can be used for validation, while `htmlspecialchars()` helps safely display user-provided text by converting special HTML characters into entities. W3Schools specifically recommends validating and sanitizing external input such as `$_POST` data.

Reference Link:

https://www.w3schools.com/php/php_form_validation.asp

https://www.w3schools.com/php/php_filter.asp

https://www.w3schools.com/php/php_form_required.asp

3. Is file writing handled safely?

Answer: Yes. When storing submitted records in CSV or JSON files, the PHP program should open/write the file carefully and check whether the operation was successful. For CSV storage, PHP provides the `fputcsv()` function. It converts an array of values into a CSV-formatted row and writes it to an open file.

Example:

```
$file = fopen("students.csv", "a");
```

```
fputcsv($file, [$name, $email]);
```

```
fclose($file);
```

The "a" mode allows new records to be added without removing existing records.

For simpler file writing, PHP also provides `file_put_contents()`. Using `FILE_APPEND` allows new content to be added instead of overwriting the existing file.

Example:

```

file_put_contents(
    "data.txt",
    $name . "," . $email . PHP_EOL,
    FILE_APPEND
);

```

Reference Link:

https://www.w3schools.com/php/func_filesystem_fputcsv.asp

https://www.w3schools.com/php/func_filesystem_file_put_contents.asp

https://www.w3schools.com/php/php_ref_filesystem.asp

4. Are success and error responses displayed clearly?

Answer: Yes. After processing the form, the PHP program should clearly tell the user whether the operation was successful or whether there was an error.

For example:

```
if (empty($error)) {  
    echo "Registration successful!";  
} else {  
    echo "Error: " . $error;  
}
```

This makes it easier for the user to understand whether the form was accepted or needs correction.

For multiple validation errors, separate error messages can be maintained for individual fields, such as \$nameErr and \$emailErr. W3Schools demonstrates this approach in its required-fields form validation example.

Reference Link:

https://www.w3schools.com/php/php_form_required.asp

A. Implementation/ Output/Result

Sign up.php

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Signup</title>
```

```
    <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<nav class="navbar">
```

```
<div class="nav-brand">StudentHub</div>
```

```
<div class="nav-right">
```

```
<a href="Home.html">Home</a>
```

```
<button id="themeToggle"
      class="theme-toggle"
      title="Toggle light/dark theme">
```



```
</button>
```

```
</div>
```

```
</nav>
```

```
<h1 align="center">Welcome to StudentHub</h1>
```

```
<br>
```

```
<center>
```

```
<fieldset class="signup-fieldset">
```

```
<form id="signupform"
```

```
    action="registration.php"
```

```
    method="POST"
```

```
    novalidate>
```

```
<h3>Student Registration</h3>
```

```
<label for="fullname">
```

```
    Full Name:
```

```
</label>
```

```
<input type="text"
```

```
    id="fullname"
```

```
    name="fullname"
```

```
    placeholder="Enter your full name"
```

```
    autocomplete="name">
```

```
<span class="error"
```

```
    id="fullNameError">
```

```
</span>
```

```
<label for="email">
```

Email:

</label>

```
<input type="email"
      id="email"
      name="email"
      placeholder="Enter your email"
      autocomplete="email">
```

```
<span class="error"
      id="emailError">
</span>
```

```
<label for="mobile">
  Mobile Number:
</label>
```

```
<input type="tel"
      id="mobile"
      name="mobile"
      placeholder="Enter 10-digit mobile number"
      maxlength="10"
      autocomplete="tel">
```

```
<span class="error"
```

```
        id="mobileError">
</span>

<label for="course">
    Course:
</label>

<select id="course"
        name="course">

    <option value="">
        -- Select Course --
    </option>

    <option value="WDF">
        Web Development Framework
    </option>

    <option value="C++">
        Programming with C++
    </option>

    <option value="DSA">
        Data Structure and Algorithm
    </option>
```

```
</select>
```

```
<span class="error"  
      id="courseError">  
</span>
```

```
<label for="year">  
  Year:  
</label>
```

```
<select id="year"  
      name="year">
```

```
<option value="">  
  -- Select Year --  
</option>
```

```
<option value="1">  
  1st Year  
</option>
```

```
<option value="2">  
  2nd Year  
</option>
```

```
<option value="3">
```

```
    3rd Year
```

```
</option>
```

```
<option value="4">
```

```
    4th Year
```

```
</option>
```

```
</select>
```

```
<span class="error"
```

```
    id="yearError">
```

```
</span>
```

```
<label>
```

```
    Gender:
```

```
</label>
```

```
<div class="gender-group">
```

```
    <label class="inline-label">
```

```
        <input type="radio"
```

```
            name="gender"
```

value="Male">

Male

</label>

<label class="inline-label">

<input type="radio"

name="gender"

value="Female">

Female

</label>

<label class="inline-label">

<input type="radio"

name="gender"

value="Other">

Other

</label>

</div>

<span class="error"
 id="genderError">

<label for="createpassword">
 Create Password:
</label>

<input type="password"
 id="createpassword"
 name="password"
 placeholder="Create the password"
 autocomplete="new-password">

<span class="error"
 id="createpasswordError">

<label for="confirmpassword">
 Confirm Password:
</label>

```
<input type="password"  
      id="confirmpassword"  
      name="confirm_password"  
      placeholder="Confirm the password"  
      autocomplete="new-password">
```

```
<span class="error"  
      id="confirmpasswordError">  
</span>
```

```
<div class="terms-group">
```

```
<label class="inline-label">
```

```
<input type="checkbox"  
      id="terms"  
      name="terms"  
      value="accepted">
```

I accept the Terms and Conditions

```
</label>
```

```
</div>
```

```
<span class="error"
      id="termsError">
</span>
```

```
<input type="submit"
      value="Register">
```

```
<p id="message"></p>
```

```
</form>
```

```
</fieldset>
```

```
</center>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

registration.php

```
<?php
```

```
// Check whether form was submitted using POST
```

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

```
// Get form values

$fullname = trim($_POST["fullname"] ?? "");
$email = trim($_POST["email"] ?? "");
$mobile = trim($_POST["mobile"] ?? "");
$course = trim($_POST["course"] ?? "");
$year = trim($_POST["year"] ?? "");
$gender = trim($_POST["gender"] ?? "");
$password = $_POST["password"] ?? "";
$confirm_password = $_POST["confirm_password"] ?? "";
$terms = $_POST["terms"] ?? "";

// Validation

$errors = [];

// Full name validation

if (!preg_match("/^[A-Za-z\s]{3,}$/", $fullname)) {

    $errors[] = "Please enter a valid full name.";

}
```

```
// Email validation
```

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
```

```
    $errors[] = "Please enter a valid email.";
```

```
}
```

```
// Mobile validation
```

```
if (!preg_match("/^[6-9][0-9]{9}$/", $mobile)) {
```

```
    $errors[] = "Please enter a valid 10-digit mobile number.";
```

```
}
```

```
// Course validation
```

```
if ($course == "") {
```

```
    $errors[] = "Please select a course.";
```

```
}
```

```
// Year validation
```

```
if ($year == "") {

    $errors[] = "Please select your year.";

}

// Gender validation

if ($gender == "") {

    $errors[] = "Please select your gender.";

}

// Password validation

if (!preg_match(
    "/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@$!%*?&]).{8,}$/",
    $password
)) {

    $errors[] =
        "Password must contain uppercase, lowercase, number and special character.";
```

```
}

// Confirm password

if ($password != $confirm_password) {

    $errors[] = "Passwords do not match.";

}

// Terms validation

if ($terms != "accepted") {

    $errors[] =

        "You must accept the Terms and Conditions.";

}

// If there are errors

if (!empty($errors)) {

    echo "<h2>Registration Failed</h2>";
```

```
echo "<ul>";

foreach ($errors as $error) {

    echo "<li>" .

        htmlspecialchars($error) .

        "</li>";

}

echo "</ul>";

echo '<a href="Sign up.php">Go Back</a>';

exit;
}

// Sanitize input

$fullname = htmlspecialchars(

    $fullname,

    ENT_QUOTES,

    "UTF-8"

);
```

```
$email = htmlspecialchars(  
    $email,  
    ENT_QUOTES,  
    "UTF-8"  
);
```

```
$mobile = htmlspecialchars(  
    $mobile,  
    ENT_QUOTES,  
    "UTF-8"  
);
```

```
$course = htmlspecialchars(  
    $course,  
    ENT_QUOTES,  
    "UTF-8"  
);
```

```
$year = htmlspecialchars(  
    $year,  
    ENT_QUOTES,  
    "UTF-8"  
);
```

```
$gender = htmlspecialchars(  
    $gender,  
    ENT_QUOTES,  
    "UTF-8"  
);
```

```
$gender,  
ENT_QUOTES,  
"UTF-8"  
);  
  
// Create registration record  
  
$registration = [  
  
    "name" => $fullname,  
  
    "email" => $email,  
  
    "mobile" => $mobile,  
  
    "course" => $course,  
  
    "year" => $year,  
  
    "gender" => $gender  
  
];  
  
// JSON file
```

```
$file = "registrations.json";

// Read existing data

if (file_exists($file)) {

    $data = file_get_contents($file);

    $registrations = json_decode(
        $data,
        true
    );

} else {

    $registrations = [];

}

// Add new registration

$registrations[] = $registration;

// Save data
```

```
file_put_contents(
    $file,
    json_encode(
        $registrations,
        JSON_PRETTY_PRINT
    )
);

// Success message

echo "<h2>Registration Successful!</h2>";

echo "<p>Welcome to StudentHub, "
    . htmlspecialchars($fullname)
    . "!</p>";

echo "<p>Your registration has been saved successfully.</p>";

echo '<a href="Sign up.php">Go Back to Sign Up</a>';

}

else {

    echo "Invalid Request.";
```

}

?>

registrations.json

[

{

"name": "Devanshi Shah",

"email": "25dce107@charusat.edu.in",

"mobile": "9876543210",

"course": "C++",

"year": "2",

"gender": "Female"

},

{

"name": "Devanshi Shah",

"email": "devanshi@gmail.com",

"mobile": "8976345673",

"course": "DSA",

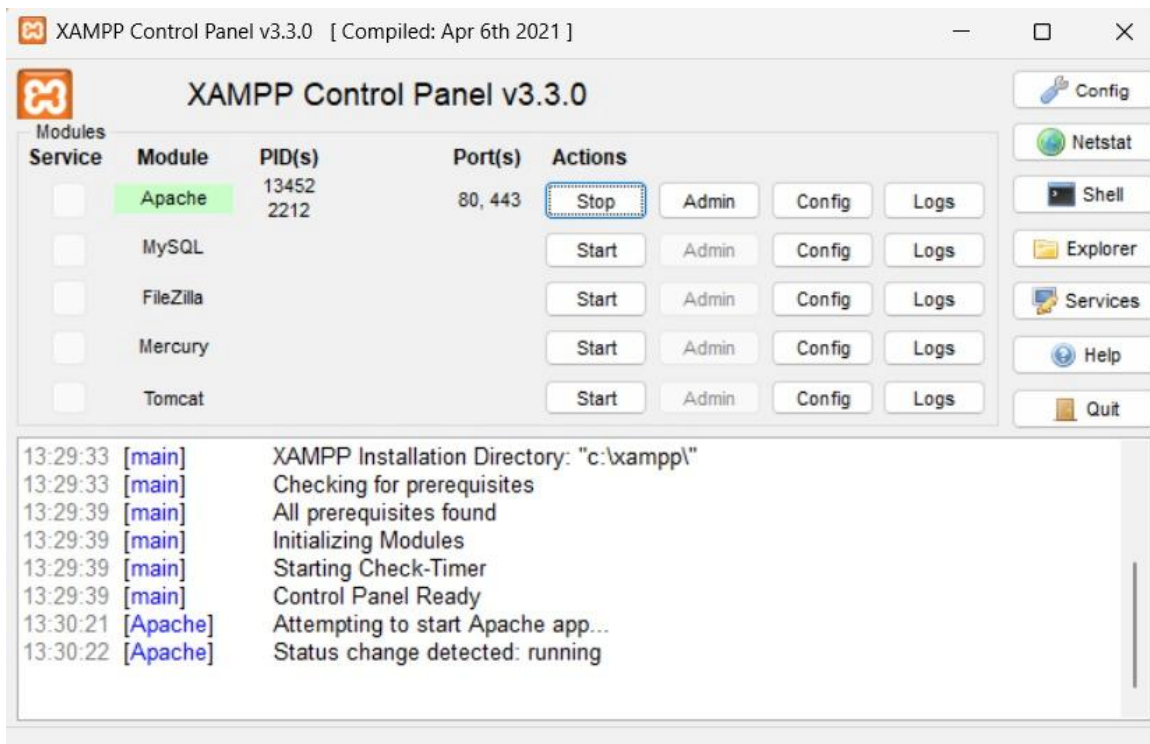
"year": "3",

"gender": "Female"

}

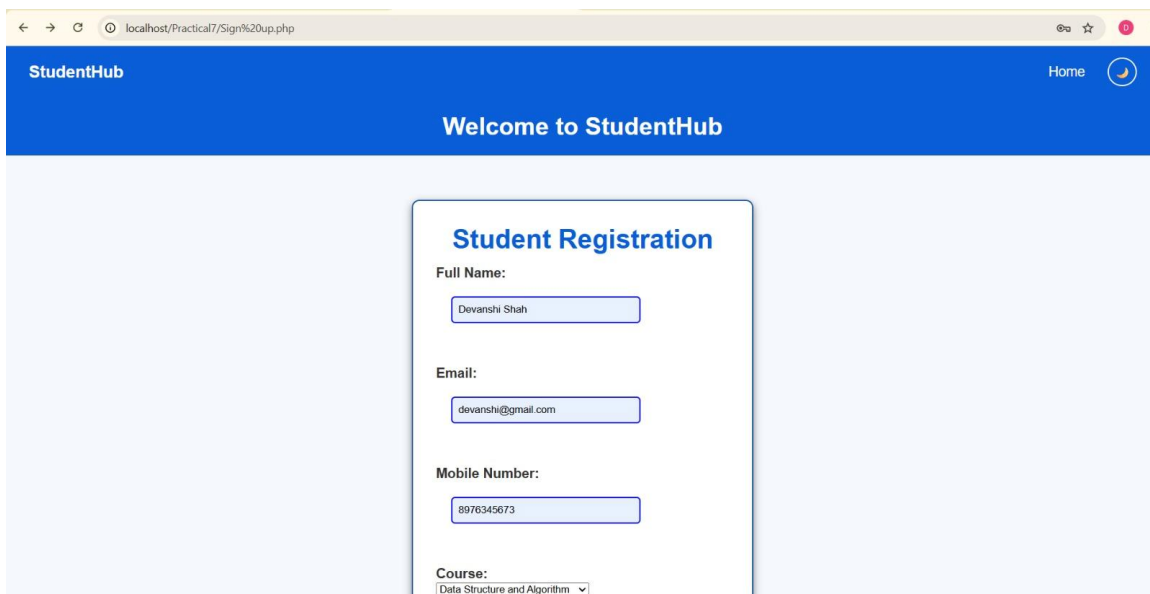
]

B. Evidence of Work (Screenshots)



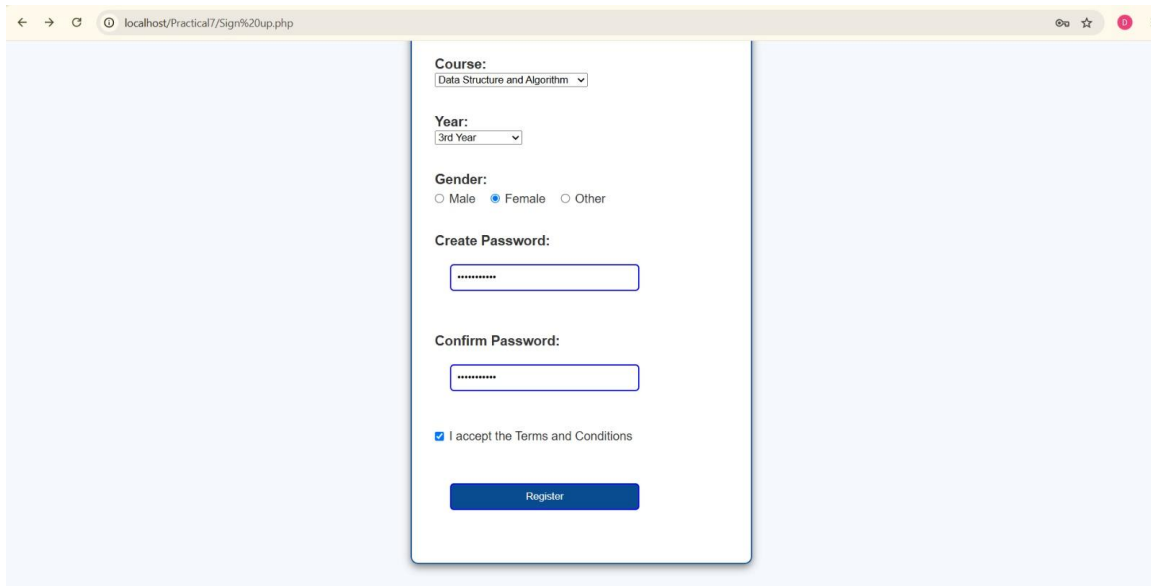
Description:

This screenshot shows the XAMPP Control Panel with the Apache server running. Apache provides the local web server environment required to execute the PHP files. The running status confirms that the PHP application can be accessed through the browser using the localhost address.



Description:

This screenshot shows the StudentHub registration form developed using PHP and HTML. The form collects student information such as Full Name, Email, Mobile Number, Course, Year, Gender, Password, and Confirm Password. The form also includes a Terms and Conditions checkbox and a Register button. This demonstrates the front-end data collection part of the practical, where user input is submitted to the PHP server for further processing and validation



The screenshot displays a web browser window with the URL `localhost/Practical7/Sign%20up.php`. The registration form is centered on a light blue background. The form fields are as follows:

- Course:** A dropdown menu with "Data Structure and Algorithm" selected.
- Year:** A dropdown menu with "3rd Year" selected.
- Gender:** Radio buttons for "Male", "Female" (selected), and "Other".
- Create Password:** A text input field with masked characters (dots).
- Confirm Password:** A text input field with masked characters (dots).
- Terms and Conditions:** A checkbox labeled "I accept the Terms and Conditions" which is checked.
- Register:** A blue button at the bottom of the form.

Description:

This screenshot shows the lower section of the registration form. The user has selected the Course, Year, and Gender and entered a password and confirmation password. The Terms and Conditions checkbox is selected before clicking the Register button. This demonstrates that all required registration fields are collected before the data is submitted to the server for processing.



Description:

This screenshot displays the “Registration Successful!” message after the form has been submitted successfully. It confirms that the registration data has been processed and saved successfully by the PHP program. The page also displays a personalized welcome message and provides a “Go Back to Sign Up” link. This demonstrates the success-response requirement of Practical 7.



Description:

This screenshot shows the registrations.json file containing the submitted registration records. The data is stored in JSON format as objects containing fields such as name, email, mobile, course, year, and gender. This confirms that the PHP form processor successfully writes the submitted information to a file-based storage system, as required in the practical.

C. GitHub Repository Link:

<https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical7>

D. Challenges Faced:

- **Handling POST data:** Understanding how to receive form values using the \$_POST superglobal was initially challenging.
- **Server-side validation:** Implementing validation for empty fields, invalid email addresses, and incorrect input formats required careful use of PHP conditions and validation functions.
- **Input sanitization:** Understanding the difference between validation and sanitization and safely displaying user-entered data using functions such as htmlspecialchars() was a challenge.
- **CSV file handling:** Managing file opening, writing records using fputcsv(), and closing the file correctly required attention to file modes and syntax.
- **JSON formatting:** Converting PHP arrays into valid JSON using json_encode() and reading JSON data using json_decode() required understanding the JSON structure.
- **Displaying appropriate messages:** It was necessary to handle both successful submissions and invalid submissions so that the user receives a clear response.
- **Testing different inputs:** Testing empty fields, invalid email addresses, special characters, and valid data helped identify and correct validation errors.

E. Learning Outcomes:

- **PHP form processing** – I learned how to receive and process form data using the POST method and the \$_POST superglobal.
- **Server-side validation** – I learned how to validate user input on the server instead of depending only on HTML or JavaScript validation.
- **Input sanitization** – I learned how to sanitize and safely display user-provided data using PHP functions such as trim() and htmlspecialchars().
- **CSV file storage** – I learned how to create and store form records in CSV files using functions such as fopen() and fputcsv().

- **JSON file storage** – I learned how to convert PHP arrays into JSON using `json_encode()` and retrieve JSON data using `json_decode()`.
- **Error handling** – I learned how to display meaningful error messages when submitted data is invalid.
- **Success handling** – I learned how to display a confirmation message after valid data has been successfully processed and stored.
- **File-based data management** – I understood how small applications can store structured form data using CSV/JSON files without requiring a database.

Faculty Remarks: (Keep this blank for faculty)